

Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

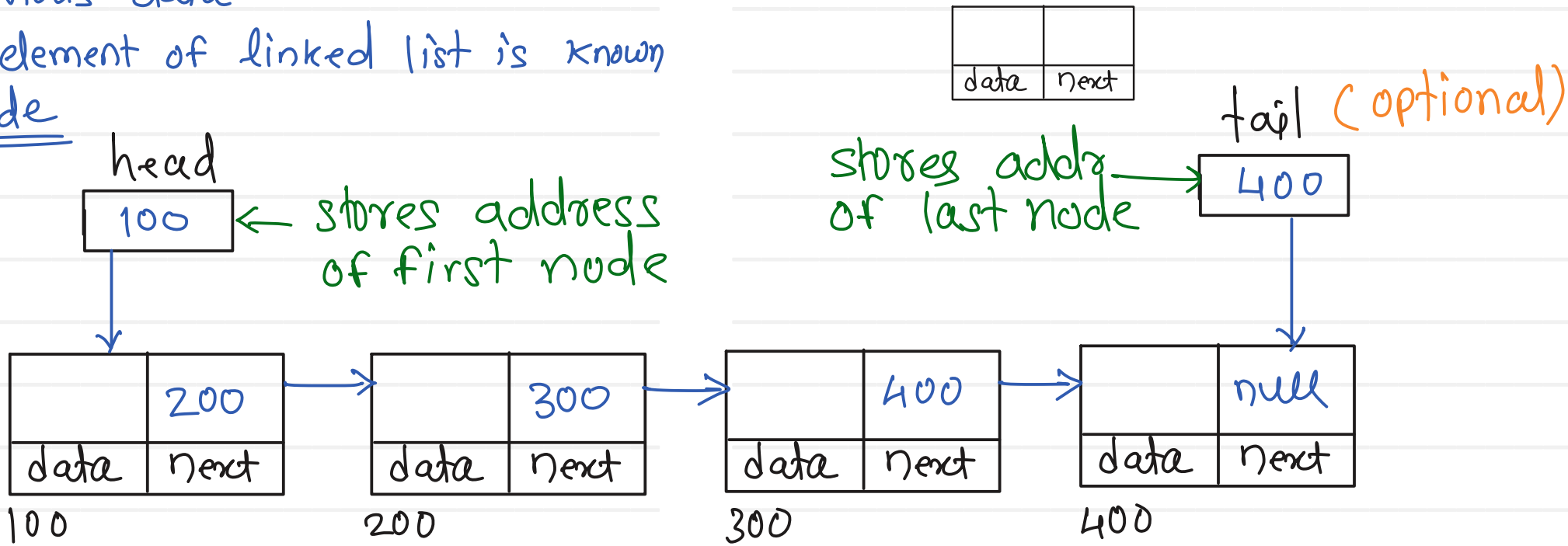
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Linked List

- linked list is a linear data structure which stores similar type of data
- address of next data is kept with its previous data
- every element of linked list is known as node

- Every node has two parts :
  1. data - actual value / content
  2. link/next - address/reference of next node



Linked List

Operations

1. Add first
2. Add last
3. Add position (insert)

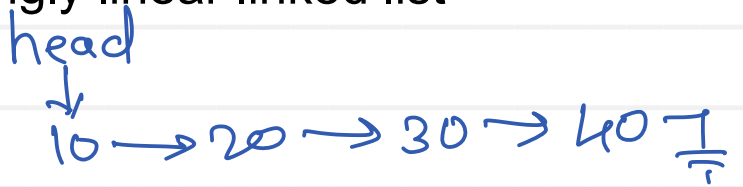
1. Delete first
2. Delete last
3. Delete position

1. Display (traverse) (forward/ backward)

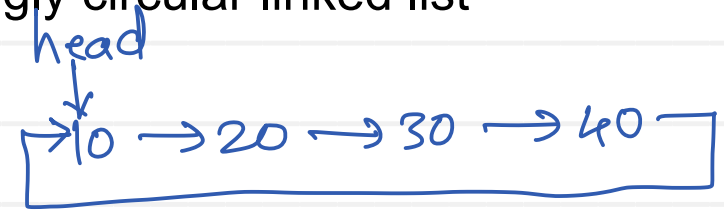
1. Search
2. Sort
3. Reverse

Types

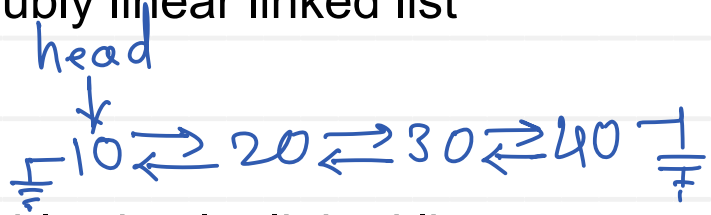
1. Singly linear linked list



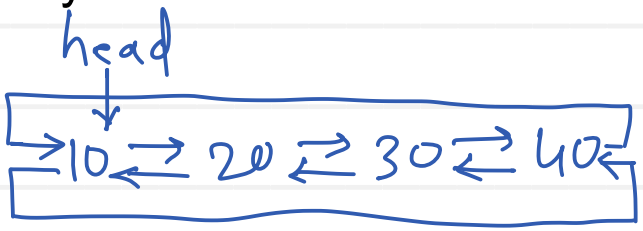
2. Singly circular linked list



3. Doubly linear linked list



4. Doubly circular linked list



Linked List

Node :
data - int, char, double, enum, class
next - reference of next node

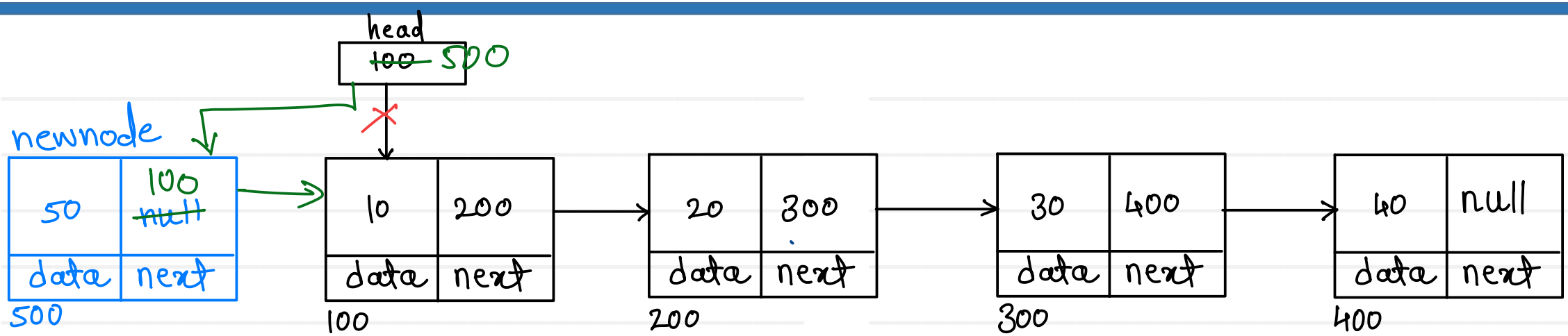
```
class Node {          <- self referential class
    int data;
    Node next;
}
```

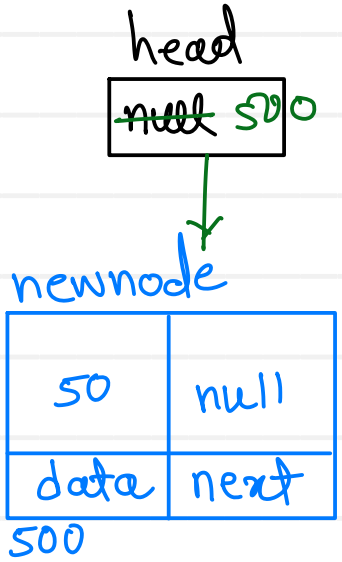
Why inner ?
- to access private fields of Node class directly into methods of LinkedList class

Why static ?
- to restrict access of private fields of LinkedList class into Node class (Non static)

```
class LinkedList {
    static class Node {
        int data;
        Node next,
        public Node() {...}
    }
    Node head, tail;
    int size;
    public LinkedList() {...}
    public addNode() {...}
    public deleteNode() {...}
    public searchNode() {...}
    public display() {...}
    public deleteAll() {...}
}
```

Singly linear Linked List - Add first



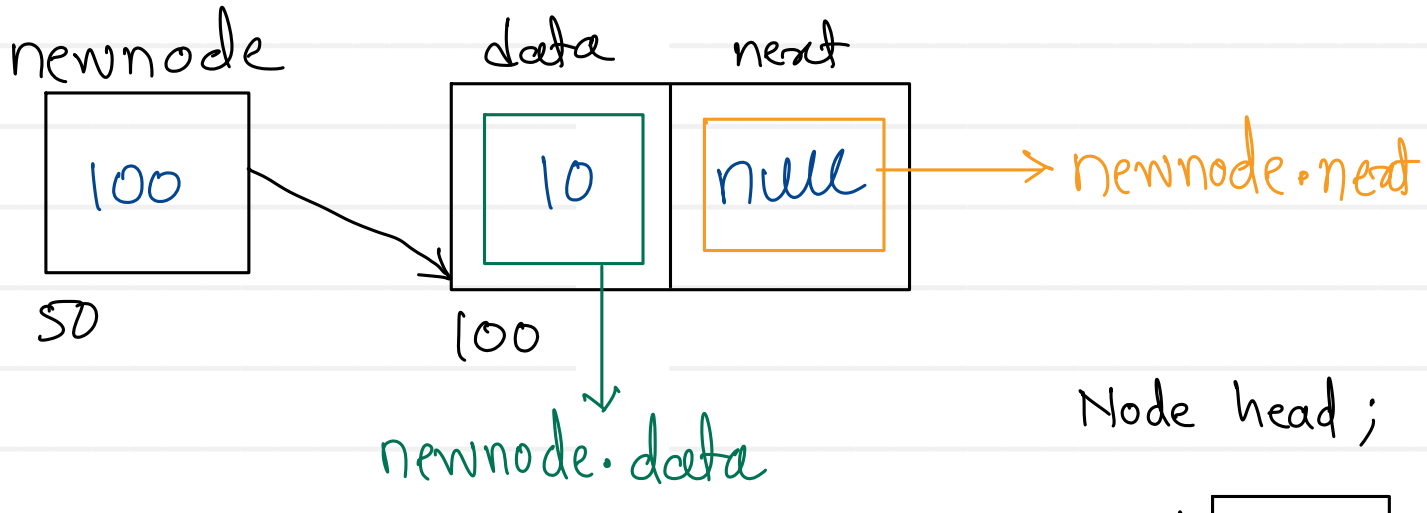



```
Node newnode = new Node(value)
newnode.next = head;
head = newnode;
```

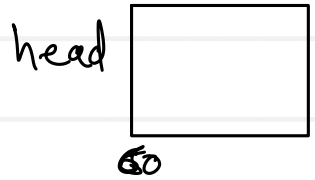
1. create newnode with given value
2. add first node into next of newnode
3. move head on newnode

$T(n) = O(1)$

Node newnode = new Node(10);



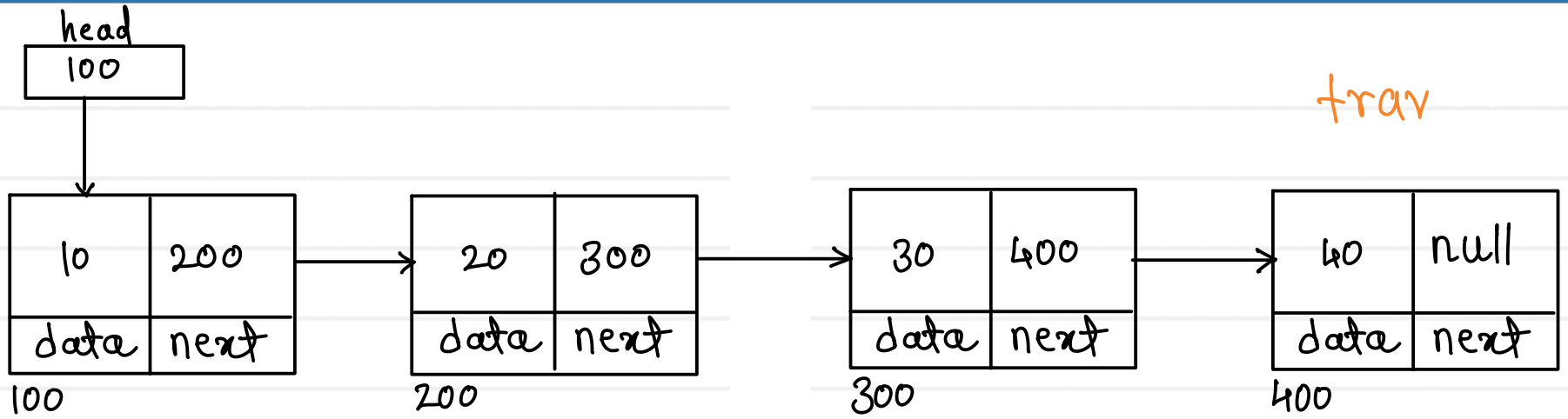
Node head;



newnode.data = 2. ← writing

2. = newnode.data ← reading

Singly linear Linked List - Display



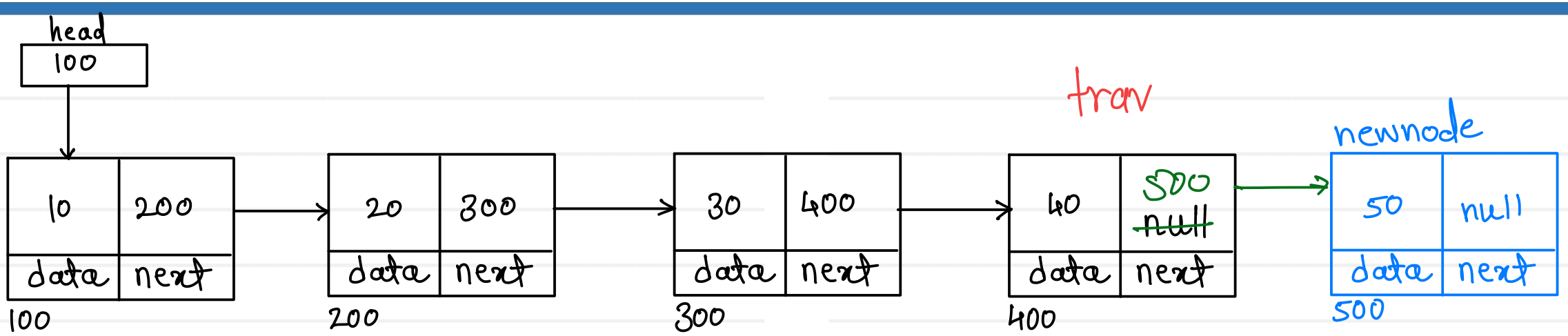
1. Create trav & start at first node
2. print/ visit current node data
3. go on next node
4. repeat step 2 & 3 till last node

$T(n) = O(n)$

```
Node trav = head;
while (trav != null){
    sysout(trav.data);
    trav = trav.next;
}
```

trav	trav.data	trav.next
100	10	200
200	20	300
300	30	400
400	40	null
null ✗		

Singly linear Linked List - Add last



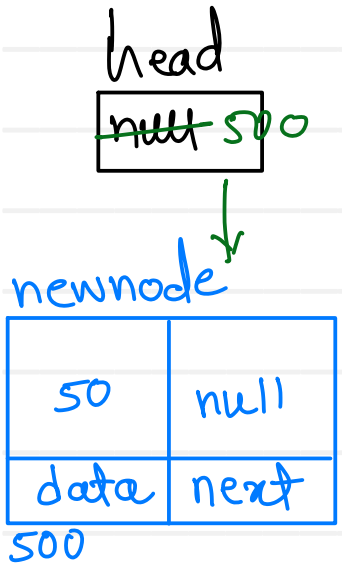
1. Create a newnode
2. if list is empty, add newnode into head itself
3. if list is not empty,
   a. traverse till last node
   b. add newnode into next of last node

$T(n) = O(n)$

```
trav = head
while( trav.next != null)
    trav = trav.next;

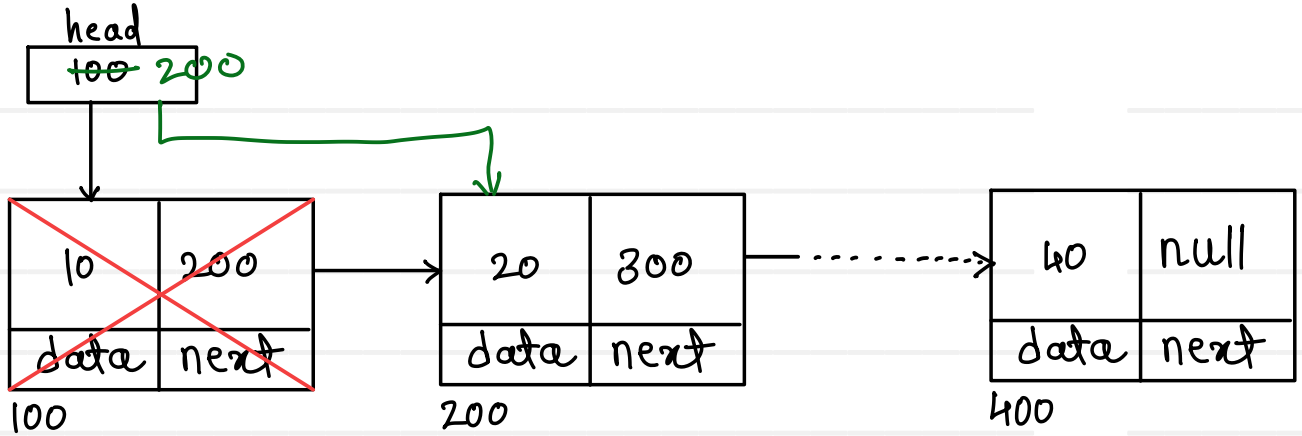
trav.next = newnode;
```

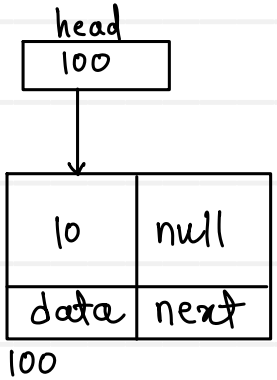
trav	trav.next
100	200
200	300
300	400
400	null ✗



O(1) → with tail reference

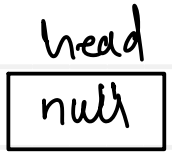
Singly linear Linked List - Delete first





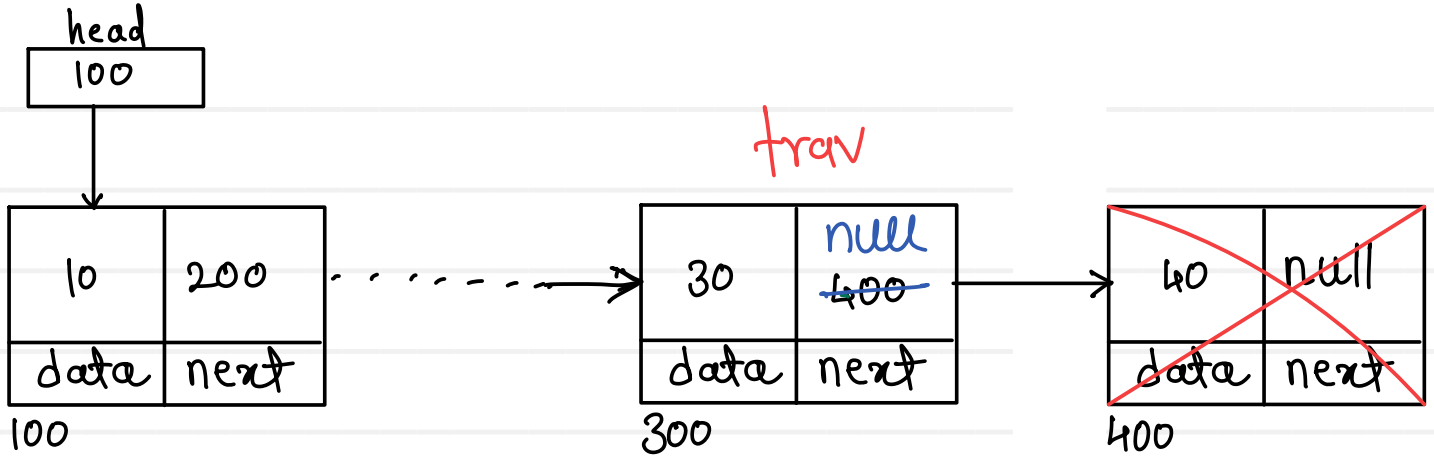
1. if list is empty, return
2. if list is not empty
   move head on second node

$T(n) = O(1)$



head = head.next
↳ null pointer exception

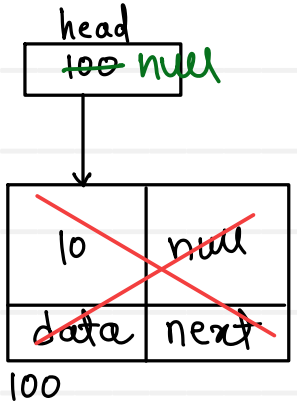
Singly linear Linked List - Delete last




```
trav = head;
while(trav.next.next != null)
    trav = trav.next;
```

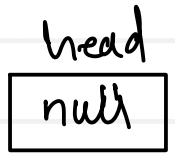
trav	trav.next	trav.next.next
100	200	300
200	300	400
300	400	null X

300 ← second last node




```
trav = head;
while(trav.next.next != null)
    trav = trav.next;
```

null pointer exception



1. if list is empty, return
2. if list has single node, delete it
3. if list has multiple nodes,
   a. traverse till second last node
   b. add null into next of second last node

$$T(n) = O(n)$$

```
Node trav = head;
while (trav != null)
    trav = trav.next;
```

→ trav = null

```
Node trav = head;
while (trav.next != null)
    trav = trav.next;
```

→ trav = last node

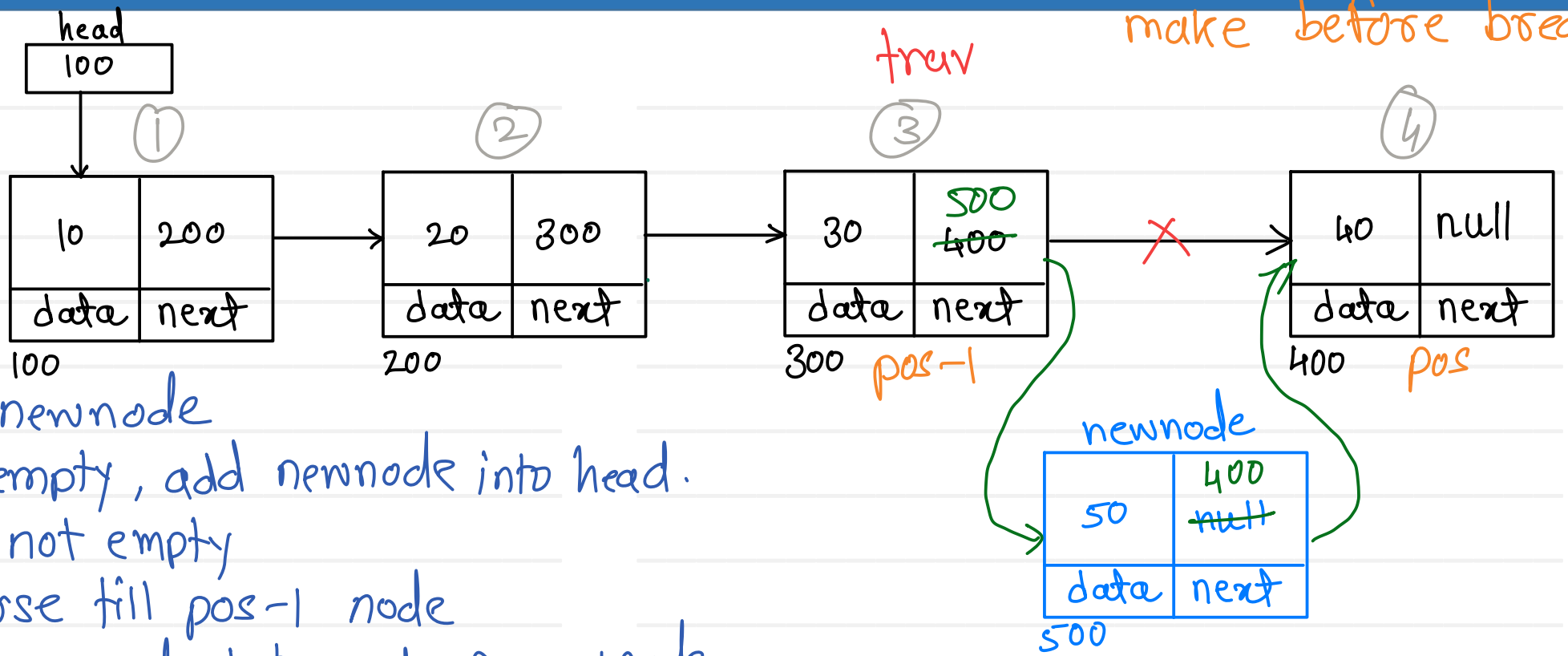
```
Node trav = head;
while (trav.next.next != null)
    trav = trav.nx -;   xl
```

→ trav = second last node

Singly linear Linked List - Add position

pos = 4

make before break



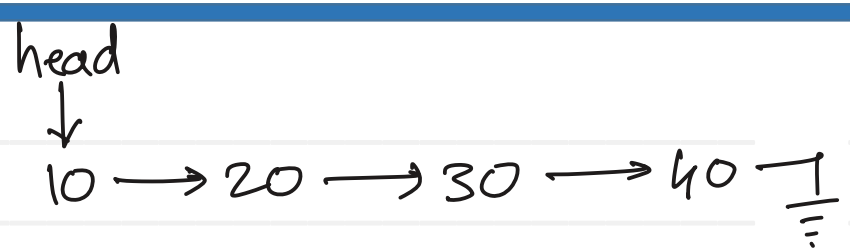
1. create a newnode
2. if list is empty, add newnode into head.
3. if list is not empty
   a. traverse till pos-1 node
   b. add pos node into next of newnode
   c. add newnode into next of pos-1 node

```
Node trav = head;
for( i=1 ; i<pos-1 ; i++)
    trav = trav.next;
```

pos = 4

trav	i	i < 3
100	1	T
200	2	T
300	3	F

$T(n) = O(n)$




```
Node trav = head;
for( i=1 ; i<pos-1 ; i++)
    trav = trav.next;

newnode.next = trav.next;
trav.next = newnode;
```

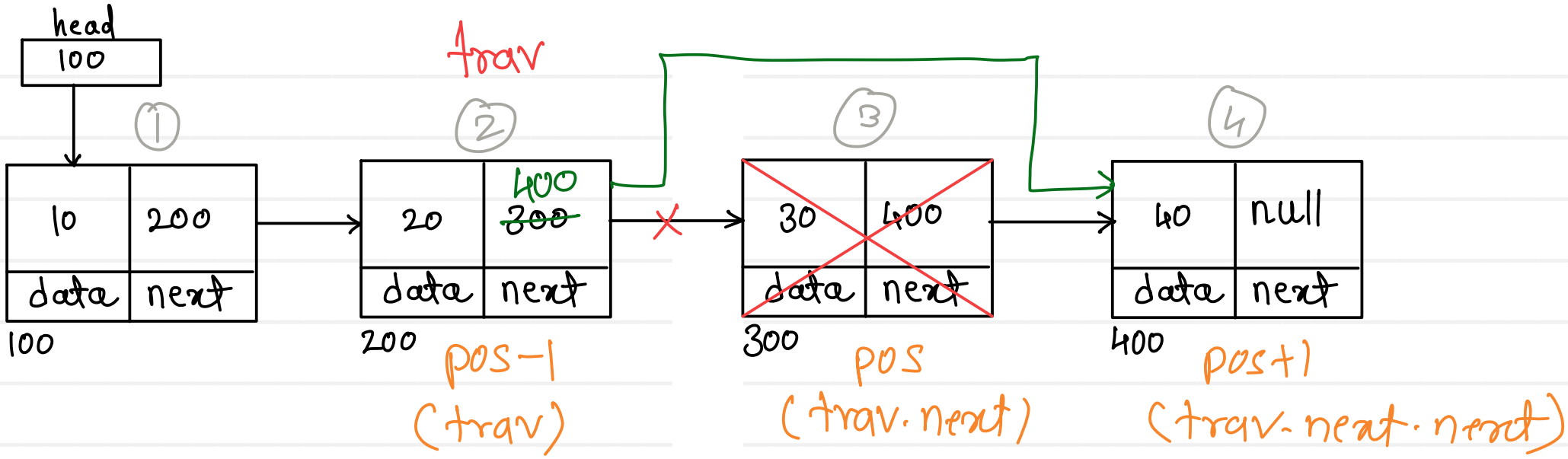
null pointer exception

pos=6

trav	i	i<5
100	1	T
200	2	T
300	3	T
400	4	T
null	5	F

Singly linear Linked List - Delete position

pos = 3

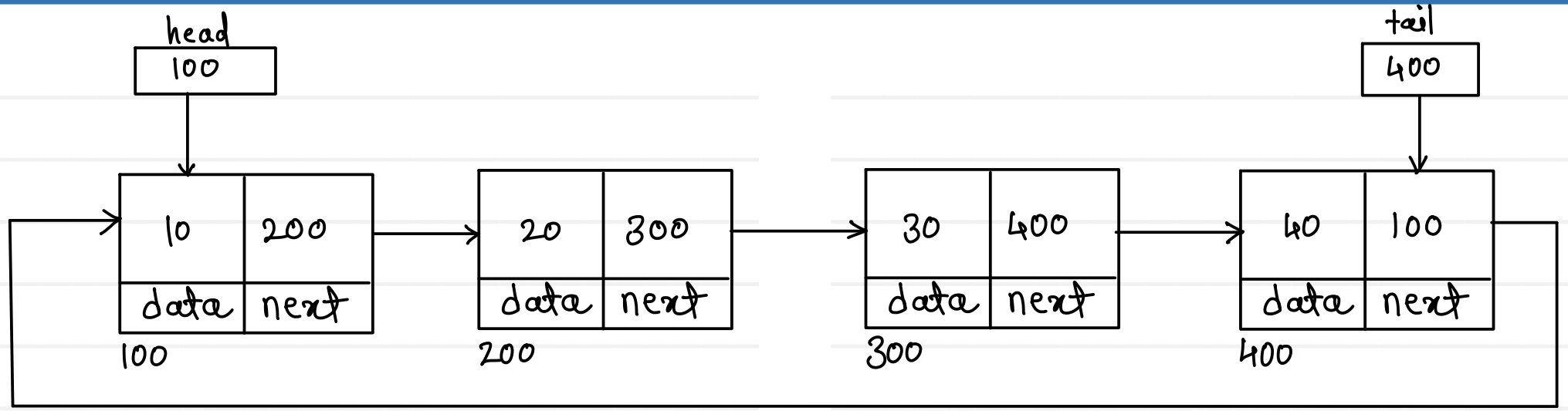


1. if list is empty, return
2. if list is not empty
   a. traverse till pos-1 node
   b. add pos+1 node into next of pos-1 node

Array Vs Linked list

Array	Linked list
• Array space inside memory is continuous	• Linked list space inside memory is not continuous
• Array can not grow or shrink at runtime	• Linked list can grow or shrink at runtime
• Random access of elements is allowed	• Random access of elements is not allowed
• Insert or delete, needs shifting of array elements	• Insert or delete, need not shifting of linked list elements
• Array needs less space	• Linked list needs more space

Singly Circular Linked List - Display



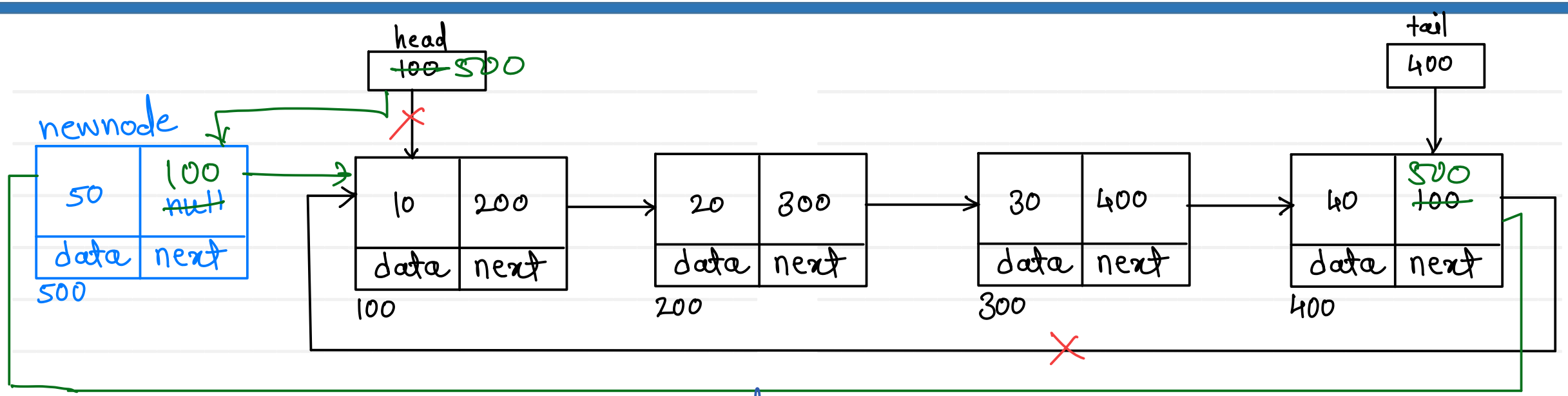
1. Create trav & start at first nod
2. print current node data
3. go on next node
4. repeate above 2 steps for each node

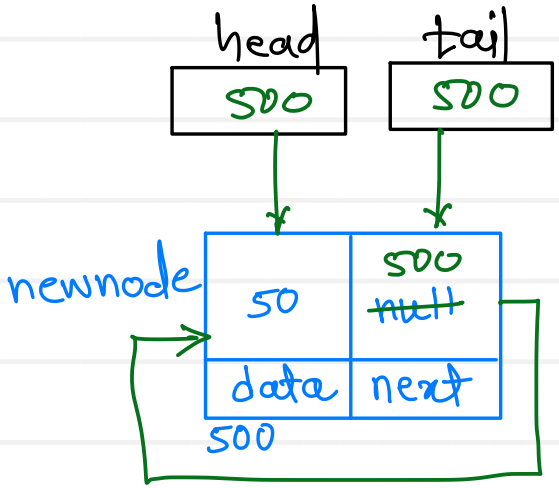
```
if(head == null)
    return;

Node trav = head;
do {
    sysout(trav.data);
    trav = trav.next;
} while( trav != head)
```

trav	trav.data
100	10
200	20
300	30
400	40
100 ✗	

Singly Circular Linked List - Add first



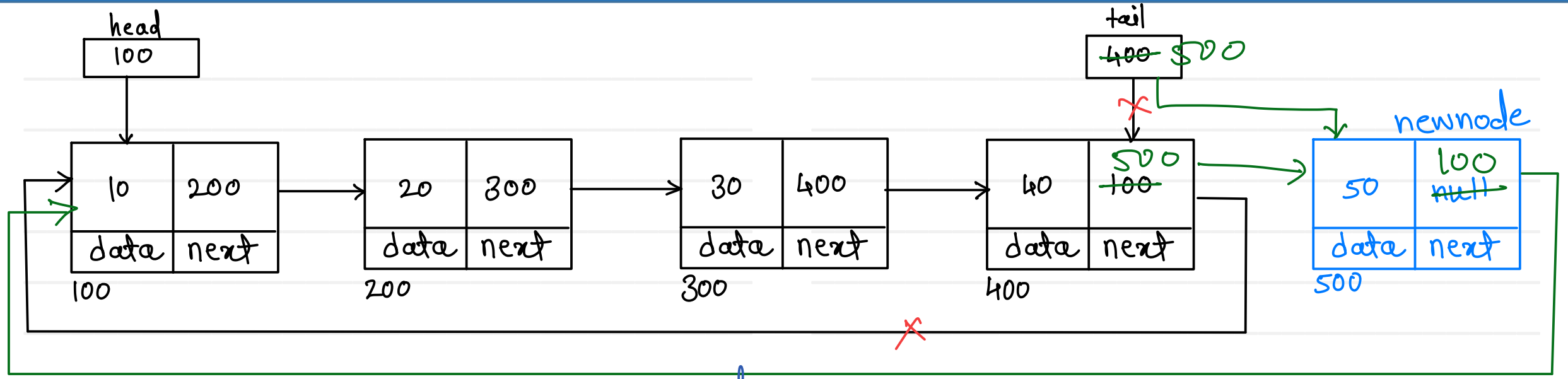


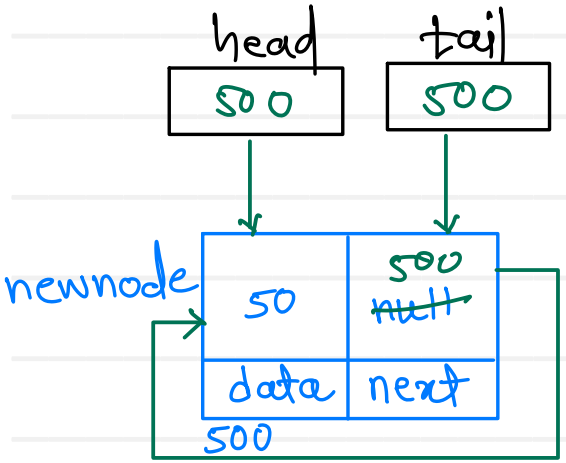
1. Create newnode
2. if list is empty,
   a. head = tail = newnode
   b. make list circular
3. if list is not empty
   a. add first node into next of newnode
   b. add newnode into next of last node
   c. move head on newnode

$T(n) = O(1)$

$O(n)$ → without tail

Singly Circular Linked List - Add last



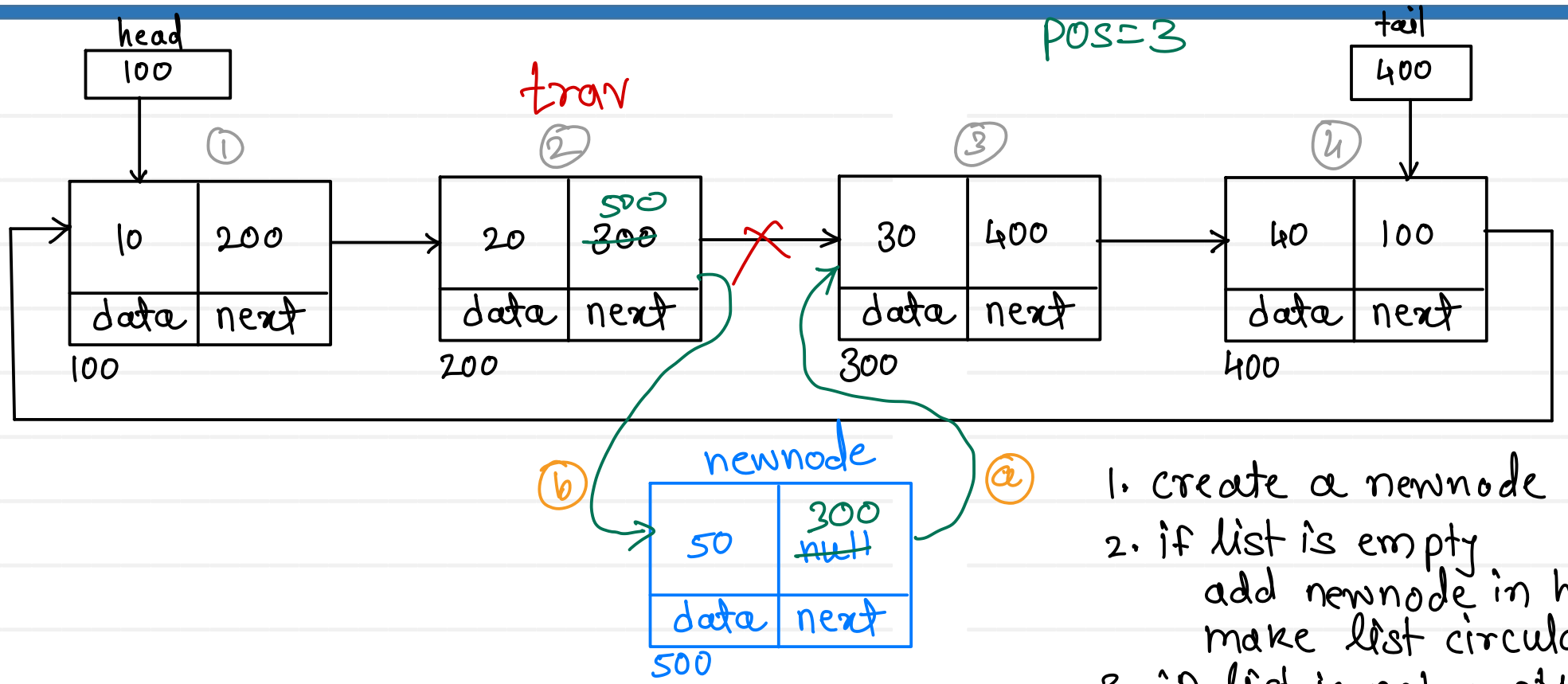


1. Create newnode
2. if list is empty,
   a. head = tail = newnode
   b. make list circular
3. if list is not empty
   a. add first node into next of newnode
   b. add newnode into next of last node
   c. move tail on newnode

$T(n) = O(1)$

O(n) → without tail

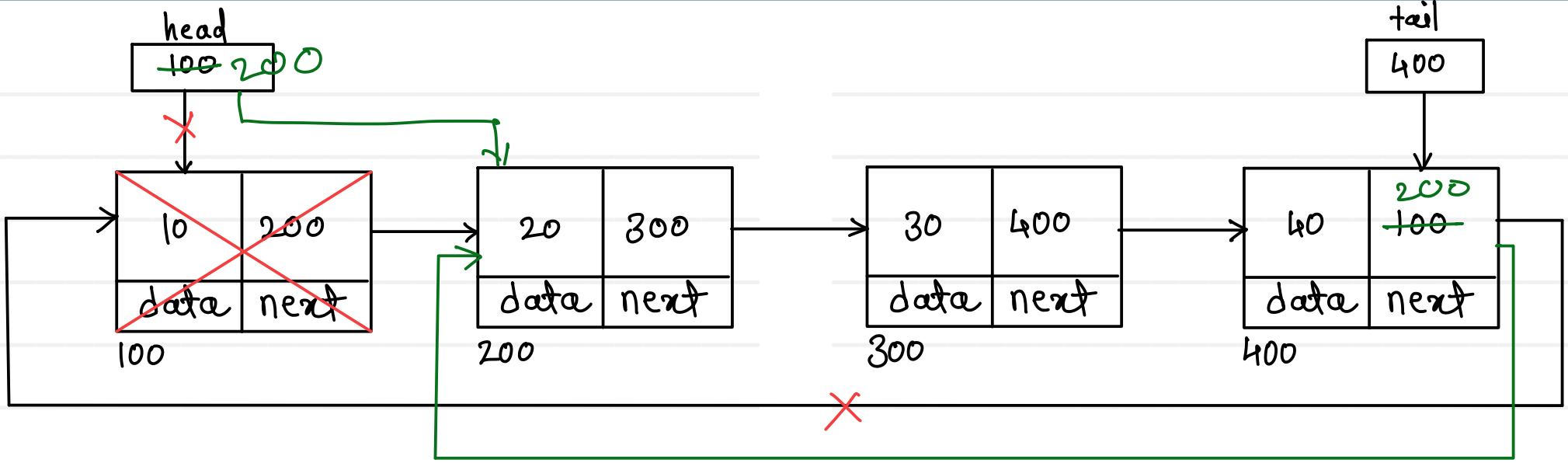
Singly Circular Linked List - Add position

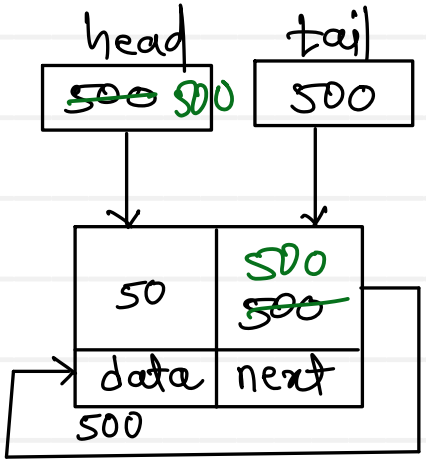


1. create a newnode
2. if list is empty
   add newnode in head & tail.
   make list circular.
3. if list is not empty
   a. traverse till pos-1 node
   b. add pos node into next of newnode
   c. add newnode into next of pos-1 node

$T(n) = O(n)$

Singly Circular Linked List - Delete first

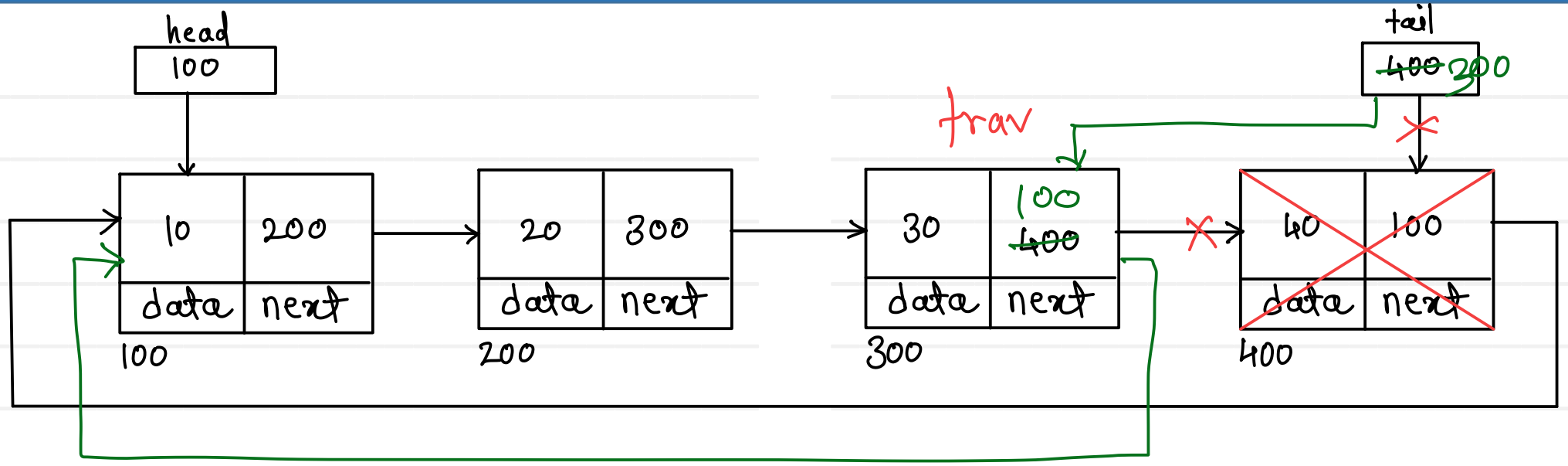





```
head = head.next;
tail.next = head
```

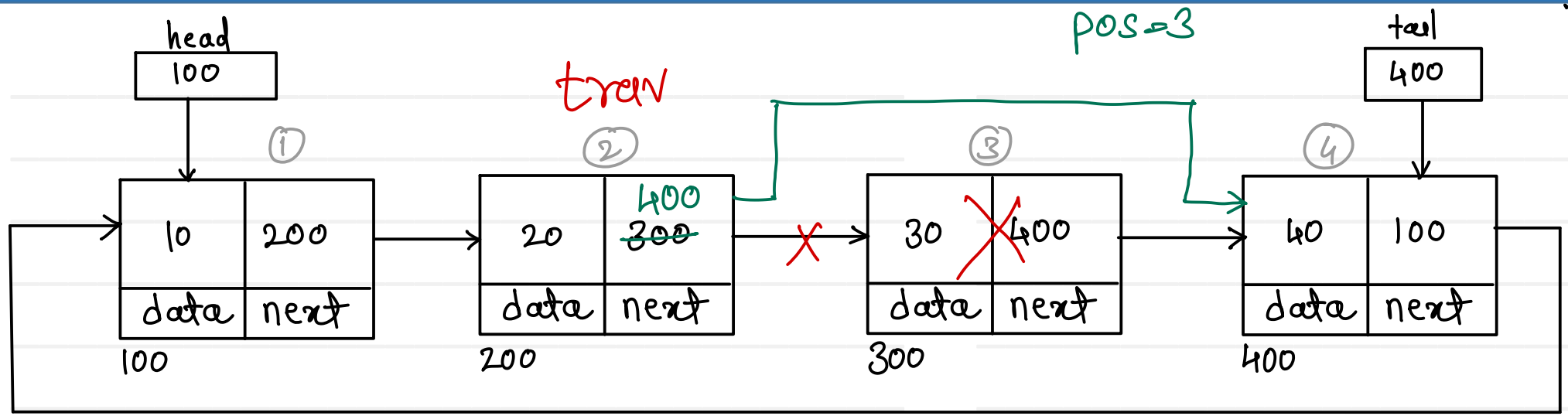
1. if list is empty, return
2. if list has single node
   head = tail = null
3. if list has multiple nodes
   a. move head on second node
   b. add second node into next of last node

Singly Circular Linked List - Delete last



1. if list is empty, return
2. if list has single node,
   head = tail = null
3. if list has multiple nodes
   - traverse till second last node.
   a. add first node into next of second last node
   b. move tail on second last node

Singly Circular Linked List - Delete position



valid positions :
1 to size
invalid positions :
pos < 1
pos > size

1. if list is empty
   return
2. if list has single node
   head = tail = null.
3. if list has multiple nodes
   a. traverse till pos-1 node
   b. add pos+1 node into next of pos-1 node

$T(n) = O(n)$